Université Abdelmalek Essaâdi
Ecole Nationale des Sciences Appliquées - Al-Hoceima
Département Mathématiques et Informatique

Filière: Data Engineering
Niveau: 2
Module: Fondements Big Data

PROJECT

Simplified Data Pipeline for a Procurement System

Proposed by: Mohamed El Marouani

December 2025

I. Introduction

The objective of this project is to initiate to the Big Data field via a real-world use case and to implement the theoretical concepts of the distributed systems. This project consists to the implementation of a **simplified data pipeline for retail stores in order to implement a procurement system**.

In the context of an academic project, we will describe in this document the context, the functional requirements, the technical environment, and the expected deliverables.

II. Context

In the grocery sector, procurement—the process of replenishing products from suppliers—is a critical activity that ensures product availability, cost control, and operational continuity. Daily purchasing decisions rely on a wide range of data: customer orders, warehouse stock levels, safety stock constraints, supplier lead times, and replenishment rules. In operational models, the freshness, availability, and reliability of the service depend directly on the accuracy and consistency of the data feeding the procurement workflow.

To support this decision-making, organizations depend on data pipelines capable of consolidating heterogeneous data sources, transforming them into usable insights, and generating automated supplier orders. However, in a pedagogical context, it is essential to simplify these mechanisms while preserving the core concepts: data integration, batch processing, centralized storage, net demand calculation, and supplier order generation.

This mini-project therefore proposes a streamlined distributed architecture—without streaming technologies—that allows students to focus on the fundamentals of data engineering and procurement modeling. Through this simplified pipeline, learners will understand how data drives procurement decisions and how a well-designed system can significantly enhance the operational efficiency of a grocery network business.

III. Functional requirements

This section describes the functional requirements of the simplified procurement data pipeline. These requirements define how the system must collect, process, integrate, and transform operational data to generate daily supplier orders for an online grocery retailer. They reflect both real-world procurement practices and the pedagogical constraints of a batch-oriented, distributed architecture.

1. Order Data Collection and Aggregation

The pipeline must ingest daily customer orders generated across multiple points of sale (POS) or delivery zones. Each POS system produces order files—typically in JSON or CSV format—which represent all customer purchases for the day. The system must collect these files, validate their structure, and store them in a centralized location. Once stored, the pipeline must aggregate order items across all stores to calculate the total demand per product (SKU). This aggregation step is essential to determine the baseline quantities that need to be replenished and to understand the daily consumption pattern across the network.

2. Warehouse Inventory Integration

In addition to customer orders, the pipeline must integrate end-of-day stock levels from one or more warehouses. These inventory snapshots contain the available quantity, the reserved quantity, and the safety stock thresholds for each product. The system must retrieve the most recent snapshot for each SKU, ensuring consistency across warehouses. By merging this inventory data with aggregated order quantities, the pipeline can compute the net stock position and determine whether additional products must be ordered from suppliers.

3. Master Data Consolidation

The pipeline must incorporate master data describing products and their supplier relationships. This includes SKU attributes, supplier identifiers, packaging constraints (such as pack size or case size), minimum order quantities (MOQ), and lead times. These datasets typically reside in a relational database such as PostgreSQL and are central to applying procurement rules. The system must validate that each SKU in the order and inventory datasets is correctly mapped to a supplier and that all mandatory fields required for procurement calculation are present.

4. Net Demand Calculation and Replenishment Logic

The core functional requirement of the pipeline is to compute the net demand for each SKU. This calculation relies on combining aggregated customer demand, available inventory, reserved stock, and safety stock requirements. The system must use the formula:

net_demand= max(0, aggregated_orders + safety_stock – (available_stock – reserved_stock))

Once the preliminary net demand is determined, the pipeline must apply supplier-specific rules. These rules include rounding the order quantity to the nearest case size, ensuring that minimum order quantities are respected, and considering ordering calendars or lead times. This step transforms raw demand signals into actionable replenishment requirements.

5. Supplier Order Generation

After identifying the required replenishment quantities, the system must generate daily supplier orders. Each supplier receives a separate file containing all SKUs assigned to them along with the computed order quantities. The pipeline must structure these export files in a standardized format—either JSON or a simplified EDI-like structure—to mimic real-world procurement interfaces. Files must be organized by date and supplier, ensuring traceability and facilitating potential reprocessing.

6. Exception Management and Data Quality Checks

The pipeline must detect and report exceptions that could compromise procurement accuracy. These include missing supplier mappings, abnormal demand spikes, missing POS files, or inconsistent stock snapshots. The system must produce an exception report summarizing all anomalies detected during the daily process. This ensures visibility over data quality issues and supports corrective actions.

7. Batch Orchestration and End-of-Day Processing

Since the project operates in batch mode, the entire pipeline—from data ingestion to order generation—must execute within a constrained time window, typically between 22:00 and 23:00 or midnight. The system must coordinate the sequencing of all tasks: collecting order files, loading them into storage, running analytical queries through the adequate framework, computing replenishments, and generating supplier order outputs. This orchestration ensures that all supplier orders are available early the next day for execution.

8. Historical Data Storage and Auditability

The system must maintain historical data for audit, analysis, and reprocessing. This includes raw POS files, processed data (such as aggregated orders or inventory snapshots), and generated supplier orders. Storing this data in a structured directory hierarchy allows rerunning the procurement process for any past date and supports further exploration or reporting activities. Auditability is a critical requirement for procurement departments to justify past orders and analyze performance.

IV. Architecture and technical stack

This section specifies the technical requirements necessary to implement the procurement data pipeline described above. It includes architectural constraints, the technical stack, data processing mechanisms, and operational considerations aligned with the pedagogical objectives of this mini-project. The goal is to provide a simplified but realistic distributed data architecture that enables students to practice core data engineering tasks without relying on advanced or real-time streaming technologies.

1. Architecture Requirements

1.1 Distributed and Modular Architecture

The system must follow a **modular**, **distributed architecture** in which each component—data ingestion, storage, transformation, and output generation—is logically separated. The architecture should simulate a real-world big data environment by combining multiple storage and compute technologies that interact through batch interfaces.

1.2 Centralized Data Lake for Batch Processing

All raw and processed files (POS orders, stock snapshots, aggregated data, exceptions, generated orders) must be stored in a distributed file system such as **HDFS**, using a clear directory hierarchy organized by date and data type.

The data lake acts as the main exchange interface between components.

1.3 Compute Engine for Analytical Queries

Since streaming technologies are excluded, the pipeline must rely on a batch-oriented analytical engine such as **Presto** (or Trino).

This engine must be able to:

- read data stored in HDFS,
- join heterogeneous datasets (orders, stock, master data),
- execute SQL-like transformations for aggregation and net demand computation.

1.4 Separation of OLTP and Analytical Layers

Master data (SKUs, suppliers, constraints) must reside in a relational database such as PostgreSQL, representing the OLTP system typically used by procurement teams. The analytical layer (HDFS + Presto) must query these tables through a connector, ensuring a realistic separation between operational data stores and analytical processing.

1.5 No-Streaming Constraint

All data movements must occur in **batch mode**, using scheduled ingestion windows or manual triggers. Protocols may include:

- flat file transfer (CSV, JSON),
- REST API simulation (optional),
- or direct copy from local to HDFS.

This respects the pedagogical constraint of avoiding Kafka, Flink, Spark Streaming, or message queues.

2. Technical Stack Requirements

2.1 Storage Layer

- **HDFS** for historical and analytical storage.
- Clear folder structure such as:
  - `/raw/orders/YYYY-MM-DD/`
  - `/raw/stock/YYYY-MM-DD/`
  - `/processed/aggregated_orders/`
  - `/processed/net_demand/`
  - `/output/supplier_orders/`
  - `/logs/exceptions/`

2.2 Relational Database

- **PostgreSQL** to store:
  - product master data,
  - supplier catalog,
  - replenishment rules (MOQ, pack size, lead times),
  - safety stock per SKU.

2.3 Query Engine

- **Presto (or Trino)** for distributed SQL queries across:
  - HDFS files,
  - PostgreSQL tables,
  - optional NoSQL data sources.

2.4 Optional NoSQL Layer

A NoSQL engine (e.g., **Cassandra**) may be used to store:

- inventory snapshots,
- time-series demand history,
- or high-frequency data retrieved from POS systems.

This is optional but useful to illustrate polyglot persistence.

2.5 File Formats and Data Standards

The system must support at least:

- **CSV** for simple representation of POS data,
- **JSON** for supplier orders,
- **optional EDI-inspired structure** for realism,
- **Parquet** (optional) for optimized Presto performance.

2.6 Scheduling and Orchestration

If you do not opt for a full workflow orchestrators like Airflow, a simpler method is required, such as:

- a Bash script with cron-style scheduling,
- a Python orchestrator script,
- or a Makefile-like execution chain.

3. Operational Requirements

3.1 Batch Execution Window

The full daily job must run in a short timeframe, typically **between 22:00 and 23:00 or midnight**, and must complete end-to-end:

- ingest files,
- run transformations,
- compute net demand,
- generate supplier orders,
- store logs and exceptions.

3.2 Fault Tolerance and Data Recovery

The pipeline must:

- handle missing or partially delivered POS files,
- produce logs and exception reports,
- avoid data overwriting by applying a write-once, append-only pattern,
- allow re-running the pipeline for a past date.

3.3 Replication & Reliability in Storage

HDFS must be configured with minimal replication (e.g., factor = 2 or 3), allowing students to understand:

- resilience concepts,
- block distribution,
- and failure tolerance.

3.4 Reproducibility and Reprocessing

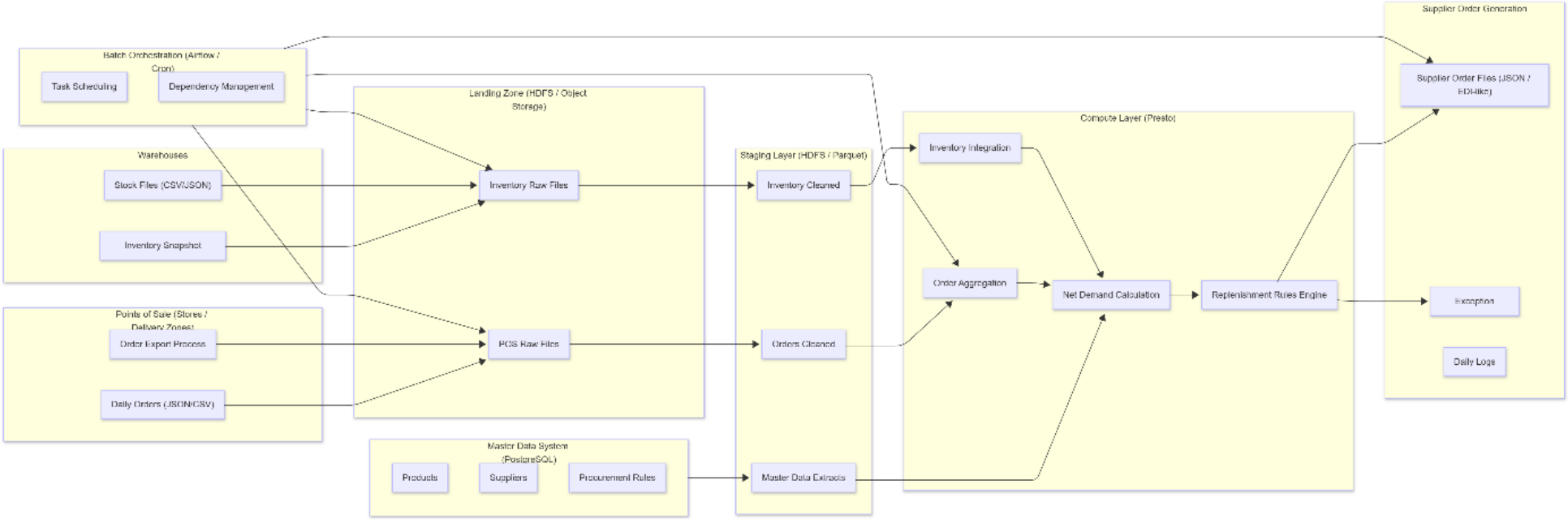
Students must be able to re-run the pipeline for any historic day by replaying the stored raw data. Outputs must be deterministic and driven only by the inputs and master data.

3.5 Scalability Considerations

Even though the dataset volume will be small, the architecture must simulate:

- distributed compute,
- multi-node query execution,
- and horizontal scaling.

The graph below sketches the main components of the architecture:



V. Data Modeling and Generation

Students must start by listing and describing the essential business entities used in the procurement process, such as:

- **Orders** (customer orders for the day)
- **Products** (items sold in the online supermarket)
- **Warehouses / Depots** (stock locations)
- **Stock Levels** (inventory snapshots)
- **Suppliers** (vendors providing replenishments)

Students must determine which data attributes are required for calculations in the pipeline, such as:

- Order quantities per product
- Current stock per product in each warehouse
- Reorder thresholds
- Supplier delivery constraints (e.g., min order quantity, lead time)

Then they will generate using faker libraries test data to simulate real operations. The objective is to create representative, not realistic, datasets.

VI. Deliverables

The deliverables of the project will be:

- Code and data: scripts, notebooks, databases, datasets, …
- Report or presentation that summarize the different parts of the project.